P.Vishwa nandan(192473015)

TASK 11 – Implement, Test and Defend the Design

Marks: 5

This is one of the most important tasks in your assignment because you must implement a working Campus Smart-Parking System and demonstrate that it works correctly through testing.

Unlike the previous tasks, here we combine the concepts learned in Tasks 1–10.

According to your assignment, we must demonstrate:

1. Normal operation: different vehicles entering and leaving.
2. Boundary cases: last free slot, zero-minute parking and fee-slab boundaries.
3. Invalid cases: unknown vehicle type, duplicate registration number and invalid slot number.
4. Calculate daily parking collection manually and compare it with program output.
5. Evaluate OOP principles and identify one limitation.
6. Compare Array, ArrayList and HashMap for finding parking tickets. CO1 JAVA

---

1. Implementation of the Smart-Parking System

Explanation

We are combining the classes developed in our previous tasks.

Our program will contain:

1. Vehicle

- Parent abstract class.
- Stores vehicle registration number.
- Calculates parking fees.

2. TwoWheeler

- Inherits Vehicle.
- Parking charge: Rs. 10 per hour.

3. Car

- Inherits Vehicle.
- Parking charge: Rs. 30 per hour.

4. ElectricCar

- Inherits Car.
- Implements the Chargeable interface.
- Charging charge: Rs. 12 per kWh.

5. ParkingPass

- Checks whether a student or staff member has a valid monthly pass.

6. ParkingTicket

- Stores ticket number, vehicle, entry time and allocated slot.
- Uses a static counter to generate unique ticket numbers.

7. ParkingFacility

- Manages parking floors and slots.
- Allocates and releases slots.
- Maintains active parking tickets.
- Calculates the total collection.

Pricing assumptions

The assignment does not specify exact parking rates. Therefore, this implementation uses the following example rates, consistent with our previous programs.

- Two-wheeler: Rs. 10 per hour.
- Car: Rs. 30 per hour.
- Electric car: Rs. 30 per hour.
- EV charging: Rs. 12 per kWh.
- Every started hour is charged fully.
- Zero-minute parking costs Rs. 0.
- Valid monthly pass holders pay no parking fee. EV charging is charged separately.

Explanation of Important Test Cases

1. Last free slot

The facility contains:

- Floor 0: Two slots.
- Floor 1: One slot.

Total capacity = 3 vehicles.

The first three vehicles receive slots successfully. The fourth vehicle is rejected because all slots are occupied.

2. Zero-minute parking

A two-wheeler enters and exits at the same time.

Parking duration = 0 minutes.

Parking fee = Rs. 0.

3. Fee-slab boundary

A car is parked for exactly 60 minutes.

Billable hours = 1.

Parking fee = 1 × 30 = Rs. 30.

For 61 minutes:

Billable hours = 2.

Parking fee = 2 × 30 = Rs. 60.

4. Invalid vehicle type

The system receives BUS as the vehicle type.

Since BUS is not supported, the system rejects the entry.

5. Duplicate registration number

The same registration number is entered while its previous ticket is still active.

The system rejects the duplicate entry.

6. Invalid slot number

The system checks floor 5, slot 0.

Since floor 5 does not exist, the position is invalid and the method returns false.

---

6. Calculate the Daily Collection Manually

The assignment asks us to calculate a small scenario manually and compare it with the program output.

Consider the following four vehicle exits.

Vehicle 1 – Two-wheeler

Duration = 0 minutes.

Parking fee = Rs. 0.

Vehicle 2 – Student with a valid monthly pass

Duration = 120 minutes.

Parking fee = Rs. 0 because the pass is valid.

Vehicle 3 – Car

Duration = 60 minutes.

Rate = Rs. 30 per hour.

Parking fee = 1 × 30 = Rs. 30.

Vehicle 4 – Electric Car

Duration = 61 minutes.

Billable hours = 2.

Parking fee = 2 × 30 = Rs. 60.

Charging consumption = 2 kWh.

Charging fee = 2 × 12 = Rs. 24.

Total EV fee = 60 + 24 = Rs. 84.

Total Collection

Rs. 0 + Rs. 0 + Rs. 30 + Rs. 84

Total = Rs. 114

Program output:

Total daily collection: Rs. 114.00

Therefore, the manually calculated amount matches the program result.

---

7. Evaluation of Object-Oriented Principles

The Smart-Parking System uses the following OOP principles.

1. Encapsulation

Private fields are used in Vehicle, ParkingTicket and ParkingPass classes.

Getter methods provide controlled access to information.

2. Inheritance

TwoWheeler and Car inherit common properties from Vehicle.

ElectricCar inherits from Car.

3. Polymorphism

The ratePerHour() method is overridden in different vehicle classes.

Java selects the correct implementation at runtime.

4. Abstraction

Vehicle is an abstract class that defines common vehicle behaviour.

Chargeable is an interface that provides charging functionality.

One Limitation

The current implementation stores active parking tickets in a fixed-size array.

Searching for a ticket requires checking the array entries one by one. This becomes slower when the number of parked vehicles increases.

Also, the ticket information is stored only in memory, so it is not preserved after the application closes.

A database and HashMap-based ticket lookup could improve the system for larger parking facilities.

---

8. Alternative Concept and Complexity Analysis

The current implementation stores active tickets in an array.

The assignment asks us to compare three data structures.

A. Array

In the current implementation, parking tickets are stored in:

ParkingTicket[] activeTickets;

To find a ticket, the system checks each array element using the registration number.

Example:

```
for (int i = 0; i < activeTickets.length; i++) {


   if (activeTickets[i] != null &&
      activeTickets[i].getVehicle().getRegNo().equals(regNo)) {


      return i;
```

```
    }
}
```

Complexity:

- Searching: O(n).
- Space: O(n).

B. ArrayList

Alternatively, active tickets can be stored using:

```
ArrayList<ParkingTicket> activeTickets = new ArrayList<>();
```

Finding a ticket:

```
for (ParkingTicket ticket : activeTickets) {

    if (ticket.getVehicle().getRegNo().equals(regNo)) {

        System.out.println("Ticket found");

        break;
    }
}
```

ArrayList provides dynamic resizing, so its capacity can grow when more tickets are added.

Complexity:

- Searching: O(n).
- Adding at the end: O(1) amortized.
- Space: O(n), including collection overhead.

C. HashMap

We can also store tickets using:

```
HashMap<String, ParkingTicket> activeTickets = new HashMap<>();
```

Here:

- Key = Vehicle registration number.
- Value = ParkingTicket object.

Finding a ticket:

```
ParkingTicket ticket = activeTickets.get("TN01AA1111");


if (ticket != null) {
   System.out.println("Ticket found");
}
```

The registration number acts as a key, so the system can directly retrieve the corresponding ticket.

Complexity:

- Searching: O(1) average case.
- Insertion: O(1) average case.
- Space: O(n), including hash table overhead.
- Worst-case searching: O(n).

D. Comparison

Data structure	Search time	Space
Array	O(n)	O(n)
ArrayList	O(n)	O(n)
HashMap	O(1) average	O(n)

Conclusion

The Campus Smart-Parking System successfully manages vehicle entry, parking slot allocation, ticket generation, fee calculation and vehicle exit. Testing confirms that normal operations, boundary conditions and invalid cases are handled correctly.

The system demonstrates encapsulation, inheritance, polymorphism and abstraction. Although the array-based implementation is suitable for a small parking facility, HashMap would provide faster ticket lookup for larger systems. Therefore, the design is suitable for the current requirements but can be improved for larger parking facilities.